MASSIVELY SCALABLE APPS LAB

LAB 3: DOCKER

PREPARED BY:
MOHAMED MEDHAT

LAB CONTENT

- Brief on Linux and Linux Commands
- Virtual Machines vs Containers
- Docker Images
- From Dockerfile to container
- Docker Compose
- Running multiple instances of a docker image
- Dockerizing Spring Boot

Linux

- Linux is a family of open-source Unix-like operating systems based on the Linux kernel, an operating system kernel first released on September 17, 1991, by Linus Torvalds.
- Since Linux is open source, you can find multiple distributions (distros) for each specific use case
  - Linux for Cybersecurity : Kali Linux
  - Linux for Embedded Systems : Embedded Linux
  - Linux for servers: ubuntu server, and more
  - Daily Drivers: Ubuntu, Mint, …..
  - Enterprise Linux: RedHat Enterprise
  - And more….

COMMON LINUX COMMANDS:

Command	Description
ls	List content in directory
cd	Change directory
pwd	Print current directory
mkdir <dirname>	Make new directory
touch <file name>	Creates a new file
echo “text”	Print text in terminal
sudo <command>	Super User Do: Execute command as admin
cat <filename>	View content of file in terminal

COMMON LINUX COMMANDS:

Command	Description
ping <url>	Send a ping request to a url
curl <url> -X <method>	Send Api request to a url
ps	Print running processes
kill <process id>	Kill/Stop a process (don’t try at home)

More commands here

https://github.com/santosh373/Linux-Basics/blob/master/linux%20cheat%20sheet.pdf

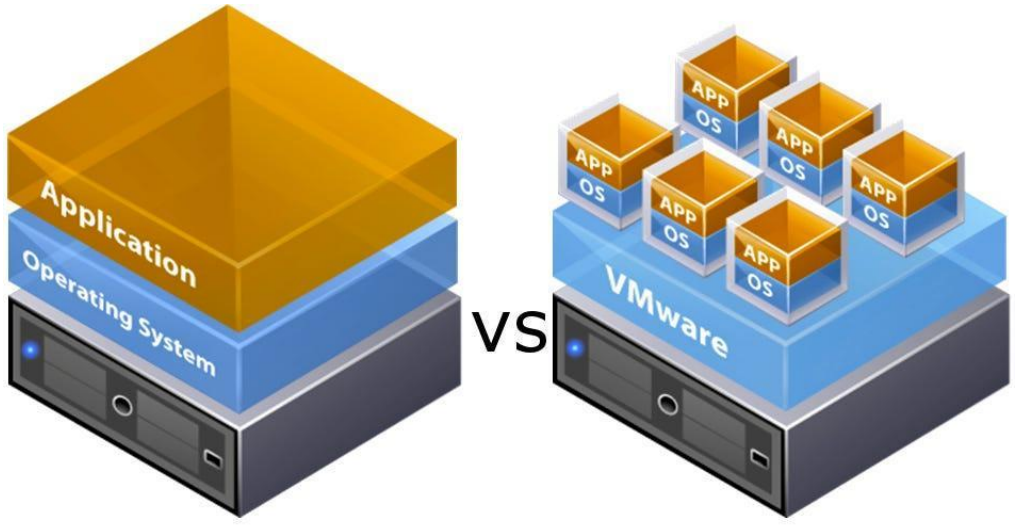
HOW WAS IT BEFORE DOCKER ?

VIRTUAL MACHINES

In its simplest form, a virtual machine is:

- a **digitized** version of a physical computer.
- can run **programs** and **operating systems**,
- store **data**,
- connect to **networks**,
- and do other computing functions.

➢ However, a VM uses entirely **virtual resources** instead of physical components.



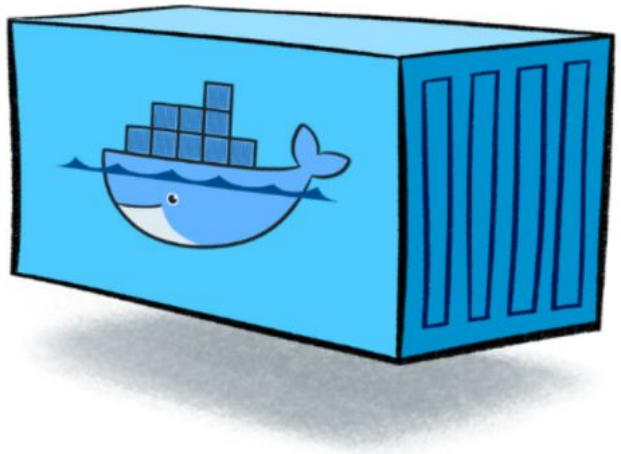
Traditional Architecture Virtual Architecture

https://cloud.google.com/learn/what-is-a-virtual-machine
https://medium.com/reverse-engineering-for-dummies/an-overview-of-virtual-machines-d409fc89ad8f

VIRTUAL MACHINES VS CONTAINERS

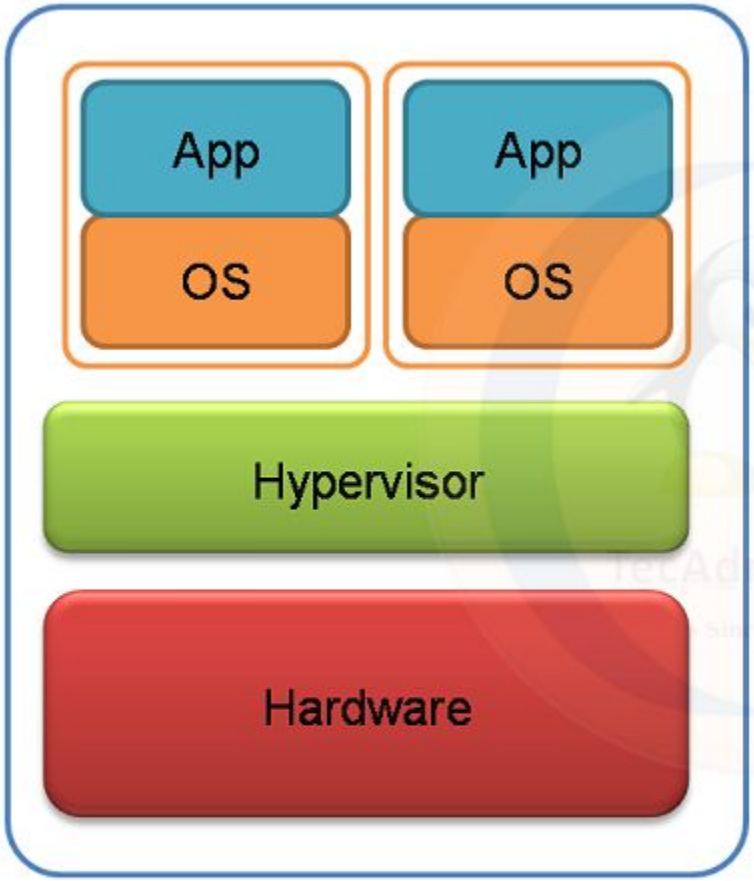
Virtual Machine

VS

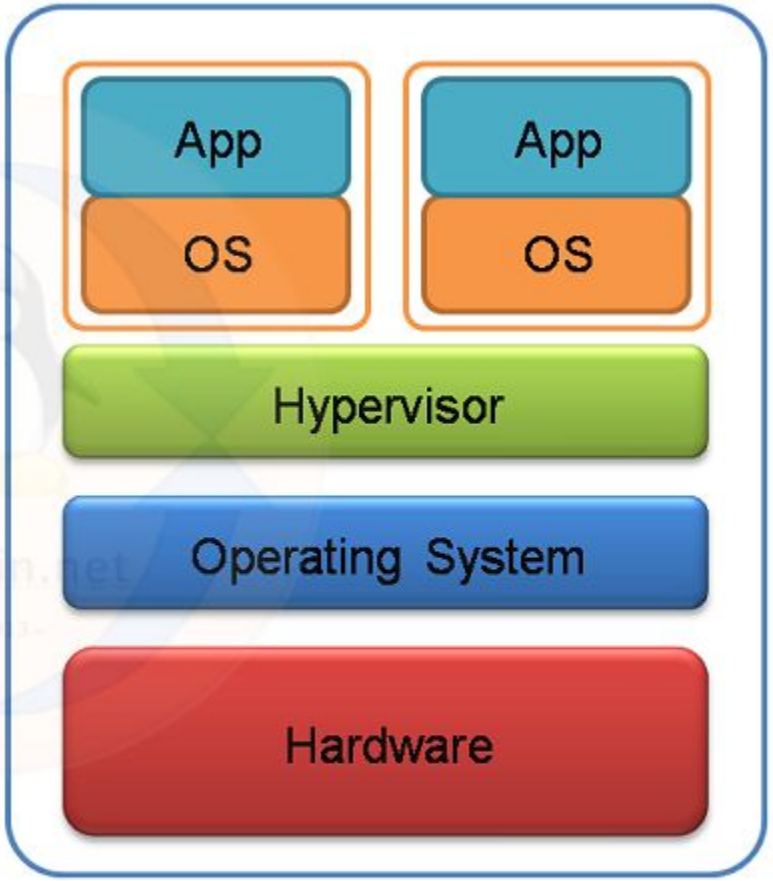


Containers

Virtual Machines



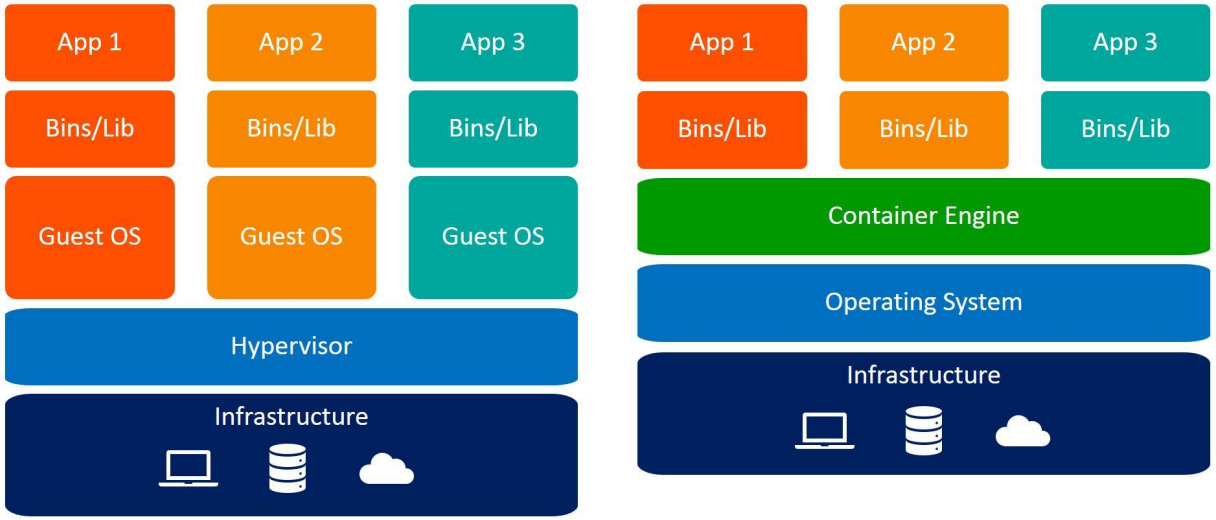
Type 1 (Bare Metal) Virtualization



Type 2 (Hosted) Virtualization

CONTAINERS

- Docker Container is a **virtual environment**,
- that **bundles** application code with all the dependencies required to run the application.
- The application runs quickly and reliably from one computing environment to another.



Virtual Machines

Containers

DOCKER IMAGES

- An **executable package** of software that includes everything needed to run an application.
- This image informs **how a container should instantiate**, determining which software components will **run** and **how**.

hub

Search Docker Hub ⌘K

Sign in | Sign up

Docker Hardened Images - Now Free

Near zero CVE hardened images with built in security, compliance, and continuous updates.
Free for all because security should be the default.

Start enterprise trial | Use for free

Generative AI

- AI Models
- MCP Servers

Trusted content

- Docker Hardened Images
- Docker Official Images
- Verified Publisher
- Sponsored OSS

Categories

- Networking

Spotlight

AI MEETS COMPOSE

Build AI Agents Faster with Docker Compose

Use the workflow you know to develop and deploy across local, cloud, and multi-cloud environments with Docker Compose.

MCP

E2B + Docker: Trusted AI

Every E2B sandbox includes direct access to Docker's MCP Catalog, a collection of 200+ tools such as GitHub, Perplexity, Browserbase, and ElevenLabs, all enabled by the Docker MCP Gateway.

SOFTWARE SUPPLY CHAIN

Secure Your Supply Chain with Docker Hardened Images

Use Docker's enterprise-grade base images: secure, stable, and backed by SLAs for Ubuntu, Debian, Java, and more. Regularly scanned and maintained with CVE remediation and long-term support.

hub

postgres ⌘K

Sign in | Sign up

Filter by

Products

- Images
- Extensions
- Helm Charts
- Compose
- AI Models

+ Show more

Trusted content

- Docker Hardened Image
- Docker Official Image
- Verified Publisher
- Sponsored OSS

Categories

- Networking

1 - 30 of 49,811 results for **postgres**.

Best match

IMAGE

PostgreSQL
dhi

The PostgreSQL object-relational database system provides reliability and data integrity.

Pulls	1M+
Stars	0
Last Updated	11 days

IMAGE + 1 MORE

postgres
Docker Official Images

The PostgreSQL object-relational database system provides reliability and data integrity.

Pulls	1B+
Stars	14802
Last Updated	about 18 hours

IMAGE

PostgreSQL Exporter
dhi

A Prometheus exporter for monitoring PostgreSQL server metrics, exposing database statistics, connection info, and...

Pulls	5.4K
Stars	0
Last Updated	about 7 hours

Give Feedback

IMAGE

CloudnativePG Ba Plugin
dhi

CloudnativePG Barman Cloud Plugin is a plugin for CloudnativePG for backing up and restoring Kubernetes...

Pulls	10K+
Stars	0
Last Updated	15 days

IMAGE

cimg/postgres
CircleCI

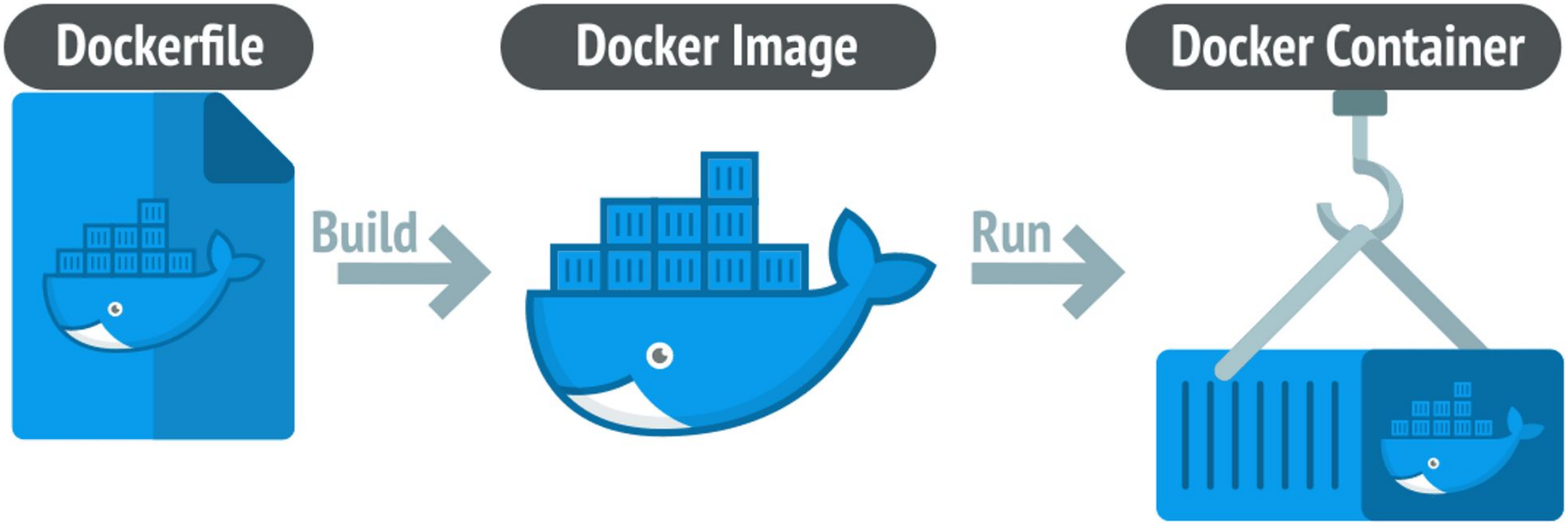
Pulls	500M+
Stars	9
Last Updated	about 1 month

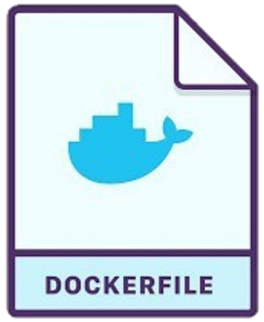
IMAGE

circleci/postgres
CircleCI

The PostgreSQL object-relational database system provides reliability and data integrity.

Pulls	500M+
Stars	35
Last Updated	about 4 years


Dockerfile
Build
Docker Image
Run
Docker Container



DOCKERFILE

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Base Image

Any custom image needs to be built on top of an existing image

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Environment Variables:
These are the environment variables that will be set inside the image that the code (project) depends on it to actually run

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Working Directory
Setting the working directory inside the container

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Copying necessary files
This command copies necessary files inside the container (later we will be using volumes for most of the files)

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Running commands
RUN command is used to run commands like compiling and installing any dependencies **while building** the image

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Working User

Defining the user in the container, similar to saying which user is logged in to the container (sometimes root needs to be set to ensure all commands are executed as admin*)

*This is dangerous

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Exposing ports

Used when we want to expose certain ports to be used outside the container, needs to be mapped when running the images

STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

Container Command
The last thing in the dockerfile is what command to run first when a container is initialised (after running)

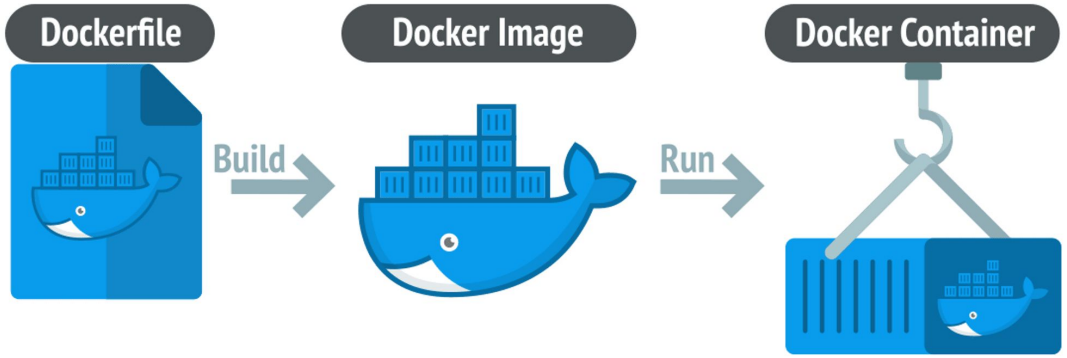
STRUCTURE OF A DOCKERFILE

```
# BASE IMAGE
FROM ubuntu:20.04

ENV PORT = 8080 \
    BASE_URL = '/api' \
    API_KEY = 'xyz123'

WORKDIR /app

COPY --from=0 /app/target/*.jar app.jar

RUN mvn clean install

# [OPTIONAL]
USER springuser

EXPOSE ${PORT}

CMD ["java","-jar", "app.jar"]
```

More commands here

https://docs.docker.com/reference/dockerfile/

BUILD YOUR DOCKER IMAGE




```
#docker build -t image_name <dockerfile_path>

docker build -t lab2_img .
```

RUN YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

The most basic way to run a docker image

RUN YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

-d stand for detached mode to run the container in the background

RUN YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

giving a name to the container

RUNNING YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

mapping ports
<host_port>:<container_port>

RUN YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

defining environment variable A to be equal to B (value)

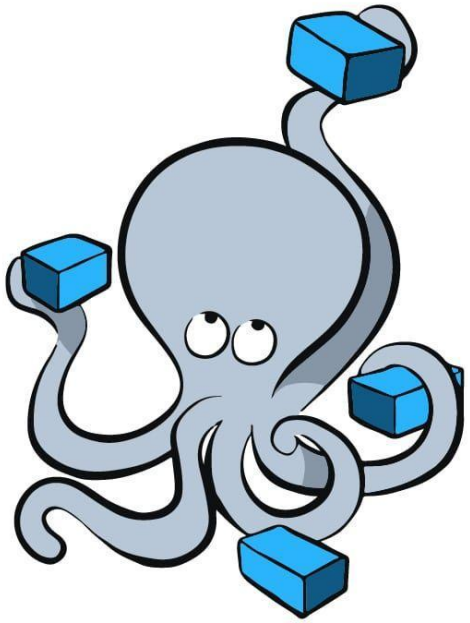
RUN YOUR DOCKER IMAGE > CONTAINER

```
# Basic
docker run image_name

# With args
docker run -d \
  --name container_name \
  -p 8080:8080 \
  -e A=B \
  img_name <arg>
```

image name, and any extra arguments

COMMON DOCKER COMMANDS

```
# Pull images from docker hub
docker pull image:tag

# Build images
docker build -t image:tag .

# Run image as container
docker run image_name
docker run -d --name container_name -p 8080:80 image_name

# Show running containers
docker ps

# Stop container
docker stop container_name

# Start a container
docker start container_name

# Remove container
docker rm container_name

# Remove all stopped containers
docker container prune

# Show images
docker images
```



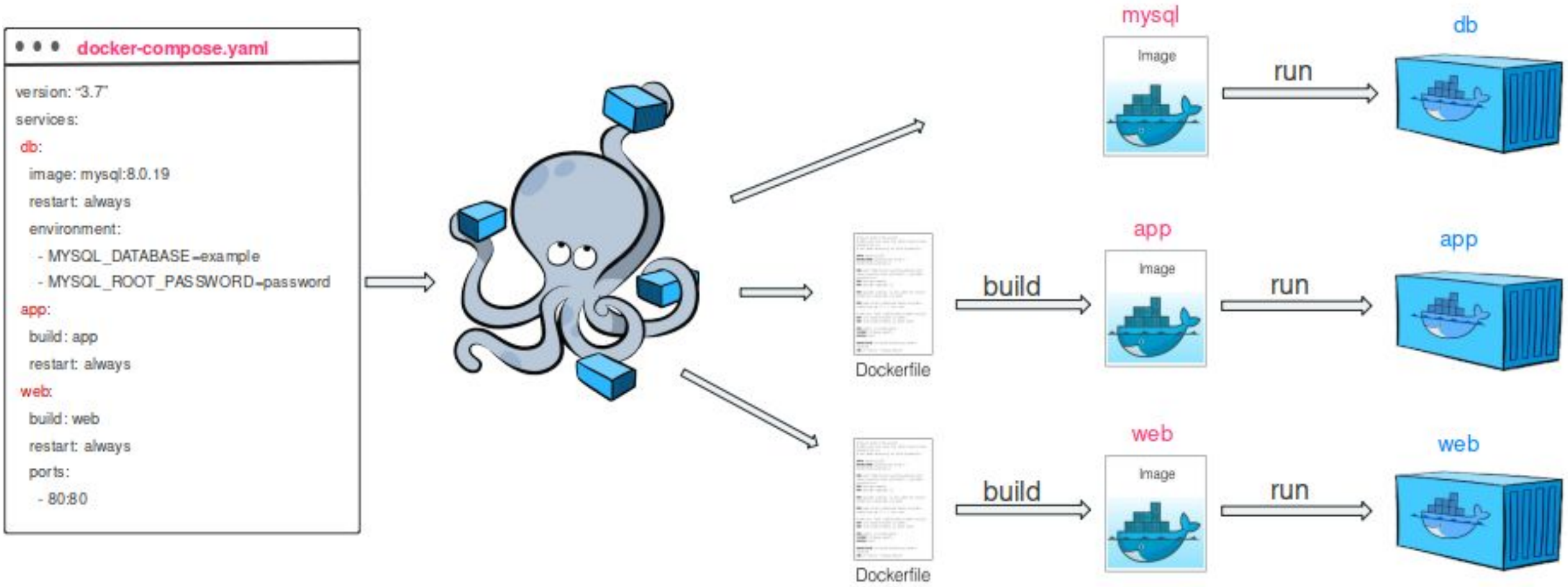
docker
Compose

DOCKER COMPOSE

- A tool for defining and running **multi-container applications**.
- It is the key to unlocking a streamlined and efficient **development** and **deployment** experience.

- It simplifies the control of your entire application stack,
  - making it easy to **manage services**, **networks**, and **volumes** in a **single comprehensible YAML configuration file**.
  - Then, with a single command, you create and start all the services from your configuration file.

- Compose works in all environments; **production**, **staging**, **development**, **testing**, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:
  - Start, stop, and rebuild services
  - View the status of running services
  - Stream the log output of running services
  - Run a one-off command on a service


docker-compose.yaml
version: "3.7"
services:
db:
image: mysql:8.0.19
restart: always
environment:
- MYSQL_DATABASE=example
- MYSQL_ROOT_PASSWORD=password
app:
build: app
restart: always
web:
build: web
restart: always
ports:
- 80:80
mysql
Image
run
db
Dockerfile
build
app
Image
run
app
Dockerfile
build
web
Image
run
web

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  Docker_Compose_lab3:
    image: lab3-img
    ports:
      - 9090:8080
    environment:
      - MYSQL_ROOT_PASSWORD=secret
```

Docker compose version

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  Docker_Compose_lab3:
    image: lab3-img
    ports:
      - 9090:8080
    environment:
      - MYSQL_ROOT_PASSWORD=secret
```

Services defined as an array of containers

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  Docker_Compose_lab3:
    image: lab3-img
    ports:
      - 9090:8080
    environment:
      - MYSQL_ROOT_PASSWORD=secret
```

Port Mapping

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  Docker_Compose_lab3:
    image: lab3-img
    ports:
      - 9090:8080
    environment:
      - MYSQL_ROOT_PASSWORD=secret
```

Defining Env Variables

USING DOCKERFILE IN DOCKER COMPOSE

In case you have not built your image yet, you can use build instead of image in the service

```yaml
version: "3.9"

services:
    app:
        build: . #path to your Dockerfile
        ...
```

HOW TO RUN MULTIPLE INSTANCES OF THE SAME SERVICE/IMAGE

1. Add Sufficient Port mappings to your docker compose, in case we need to create 10 instances we need to add 10 port mappings for example from 4040 to 4049

```yaml
services:
    app:
        ports:
            - "4040-4049:8080"
            ...
```

2. Use the --scale argument when running docker compose up

```
docker compose up -d --scale app=10
```

DOCKER COMPOSE COMMANDS

```
# Start services
docker compose up

# Start in detached mode
docker compose up -d

# Stop services
docker compose down

# Stop and remove volumes
docker compose down -v

# View logs
docker compose logs

# Scale service
docker compose up -d --scale webapp=3

# List containers
docker compose ps
```

DOCKERIZING SPRING BOOT

```
# Base image
FROM openjdk:25-ea-4-jdk-oraclelinux9

# Set working directory
WORKDIR /app

# Copy the JAR file
COPY target/*.jar app.jar

# Expose the port your app runs on
EXPOSE 8080

# Command to run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build the project and ignore the tests
mvn clean package -DskiptTests

# Build docker image
docker build -t lab2-img .

# Run docker container
docker run -d --name lab2-container -p 8080:8080 lab2-img
```

How to access ENV variables inside your code

```java
@Value("${MY_VARIABLE}")
private String myVariable;

// With default value
@Value("${MY_VARIABLE:defaultValue}")
private String myVariableWithDefault;
```

```java
private final Environment env;

@Autowired
public ClassConstructor(Environment env){
  this.env = env;
}
```

THANK YOU